React: temas explicados

Guía complementaria del Parcial 1 · Basada en las seis hojas compartidas. Explicaciones desde lo básico, ejemplos y errores frecuentes.

1. Cómo empieza una aplicación

Componentes, JSX y archivos principales

React es una biblioteca de JavaScript para construir interfaces mediante **componentes**: piezas que describen una parte de la pantalla. Un componente puede ser un formulario, un contador o una lista. En estos ejercicios se usan funciones que devuelven JSX.

JSX permite escribir etiquetas dentro de JavaScript. Las llaves insertan expresiones, como `{nombre}`. Los nombres de componentes empiezan con mayúscula; las etiquetas HTML, con minúscula. Las etiquetas deben cerrarse y los elementos hermanos se agrupan en un contenedor o fragmento.

Archivo	Responsabilidad en el ejemplo
`index.html`	Contiene el elemento root y carga main.jsx mediante un script de tipo module.
`main.jsx`	Es el punto de entrada: importa App y conecta React con el elemento del DOM.
`App.jsx`	Define el componente principal, que puede incluir otros componentes.

```
// Dentro de index.html
<div id="root"></div>
<script type="module" src="/src/main.jsx"></script>

// main.jsx
import { createRoot } from 'react-dom/client';
import App from './App.jsx';
createRoot(document.getElementById('root')).render(<App />);

// App.jsx
export default function App() {
  const nombre = 'Braian';
  return <h1>Hola, {nombre}</h1>;
}
```

El navegador carga el HTML, ejecuta el módulo de entrada y React muestra App dentro de root. El nombre `root` es una convención: lo esencial es que el id del HTML coincida con el buscado en JavaScript.

Conexión con el parcial. main.jsx monta la aplicación; App.jsx describe su interfaz. El script no es el contenedor donde aparece la aplicación.

Para ampliar: Componentes · JSX · createRoot

2. DOM, renderizado y estado

Qué cambia en la memoria y qué cambia en la pantalla

El **DOM real** es el árbol de elementos que administra el navegador. El **Virtual DOM** es una representación de la interfaz en memoria. Al renderizar, React ejecuta los componentes y calcula la nueva interfaz; al confirmar los cambios (commit), modifica el DOM real donde corresponde.

La **reconciliación** determina cómo actualizar la interfaz respecto de la anterior. El diffing es la comparación de sus estructuras. Un nuevo render no significa recargar la página ni reemplazar todos sus elementos.

useState: la memoria del componente

```
const [contador, setContador] = useState(0);
// contador: valor de este render; 0 es el valor inicial.
// setContador: solicita un nuevo valor y otro render.
function incrementar() {
  setContador(prev => prev + 1);
}
```

El estado se conserva entre renders mientras el componente mantiene su identidad en la interfaz. Una variable local como `let normal = 0` se declara otra vez cada vez que se ejecuta la función. Cambiarla no solicita una actualización visual.

Por qué el ejercicio muestra resultados distintos

```
let normal = 0;
const [estado, setEstado] = useState(0);
function incrementar() {
  normal = normal + 1;
  setEstado(estado + 1);
  console.log('Normal:', normal);
}
```

Primer clic	Qué sucede
En el manejador	normal pasa a 1 y la consola imprime Normal: 1.
En el siguiente render	React conserva estado = 1, pero let normal = 0 se ejecuta otra vez.
En pantalla	Normal: 0 y Estado: 1.

El estado es una instantánea

El setter no cambia la variable del render que ya se está ejecutando. Si el valor es 0, llamar a `setEstado(estado + 1)` y luego imprimir `estado` en ese mismo manejador imprime 0. El 1 pertenece al siguiente render. Para encadenar incrementos, usar `setEstado(prev => prev + 1)`: cada actualización recibe el resultado anterior.

Conexión con el parcial. No asignes directamente al estado. Usa su setter. En el contador original, la variable normal vuelve a 0 en cada render.

Para ampliar: Render y commit · Memoria · Instantáneas · Actualizaciones

3. Formularios controlados

Cómo conectar lo que escribe el usuario con el estado

En un **input controlado**, React determina el valor visible mediante el estado. `value` lleva el estado al campo y onChange lleva lo escrito de vuelta al estado. Cada edición solicita un render que mantiene ambos sincronizados.

El recorrido de una tecla

El usuario escribe → se ejecuta onChange → `e.target.value` contiene el texto → el setter guarda el nuevo valor → React renderiza el input con ese valor.

```
import { useState } from 'react';

export default function Formulario() {
  const [email, setEmail] = useState('');
  function enviar(e) {
    e.preventDefault();
    console.log('Email:', email);
    setEmail('');
  }
  return (
    <form onSubmit={enviar}>
      <label>
        Email:
        <input type="email" required value={email}
          onChange={(e) => setEmail(e.target.value)} />
      </label>
      <button type="submit">Enviar</button>
    </form>
  );
}
```

onChange y onSubmit tienen tareas diferentes

onChange actualiza el dato mientras se edita. **onSubmit** procesa el envío. En este patrón, `e.preventDefault()` cancela el envío nativo del navegador para evitar navegar o recargar la página. No envía datos a un servidor por sí mismo; el ejemplo solo los imprime.

`type="email"` permite la comprobación nativa del formato, y `required` exige un valor. Esa validación puede impedir que se dispare el envío si el campo es inválido. `preventDefault()` no valida ni limpia: el ejemplo limpia mediante `setEmail('')`.

Controlado	No controlado
Usa value y actualiza el estado con onChange.	El navegador mantiene el valor del campo.
El estado determina el valor actual.	defaultValue solo establece el valor inicial.

Conexión con el parcial. Con value pero sin actualizar el estado al escribir, el texto no se conserva. La otra falla del parcial es omitir preventDefault en el envío.

Para un checkbox se usa checked y e.target.checked; para texto, value y e.target.value. Inicializar texto con una cadena vacía evita empezar con undefined.

Para ampliar: Inputs · Envío de formularios

4. useEffect y dependencias

Cuándo ejecutar trabajo asociado a la interfaz

`useEffect` permite sincronizar un componente con sistemas externos, como una API o un temporizador. Se ejecuta después de que React confirma un render. **Montar** es incorporar el componente a la interfaz; **desmontar** es retirarlo.

Forma	Comportamiento habitual
`useEffect(fn, [])`	Se ejecuta al montar. No se repite por cambios de estado durante ese montaje.
`useEffect(fn, [provincia])`	Se ejecuta al montar y cuando provincia cambia respecto del render anterior.
`useEffect(fn)`	Se ejecuta después de cada render confirmado.

El arreglo debe incluir los valores reactivos que utiliza el efecto, por ejemplo estados o props. No se elige vacío solo para que el efecto “no moleste”: debe representar de qué datos depende su trabajo.

Ejemplo: detectar una provincia seleccionada

```
useEffect(() => {
  if (!provincia) return;
  console.log('Consultar localidades de:', provincia);
  // Aqui iria la consulta a la API de localidades.
}, [provincia]);
```

Si provincia empieza vacía, el efecto se ejecuta, pero la condición evita consultar. Cuando el usuario elige una provincia, el efecto vuelve a ejecutarse. Si se cambia a `[]`, no reaccionará a selecciones posteriores.

Cómo se forma un ciclo de peticiones

Sin arreglo de dependencias, el efecto del parcial consulta usuarios después de cada render. Al recibir datos llama a los setters, estos provocan otro render y el efecto inicia otra consulta. El problema surge de combinar la repetición del efecto con sus actualizaciones de estado.

Limpieza y StrictMode

```
useEffect(() => {
  const id = setInterval(() => console.log('tick'), 1000);
  return () => clearInterval(id);
}, []);
```

La función devuelta es la **limpieza**: libera recursos antes de volver a ejecutar un efecto por dependencias cambiadas y al desmontar. En desarrollo, StrictMode puede agregar un ciclo de configuración y limpieza para detectar errores. Por eso [] no significa “una sola vez para siempre”.

Conexión con el parcial. Para el parcial, [] corresponde a la carga al montar; [provincia], a la carga al montar y al cambiar la provincia. Sin arreglo, las actualizaciones del ejemplo generan el ciclo.

Para ampliar: Referencia de useEffect · Sincronización y limpieza

5. Consultas a una API

fetch, async/await, JSON y estados de carga

Una **API** permite solicitar datos a un servicio. Una consulta de red tarda un tiempo y puede fallar. Por eso se trata como una operación **asíncrona**: la aplicación puede seguir atendiendo otras tareas mientras espera.

Una función async devuelve una promesa. Dentro de ella, `await` pausa la continuación de esa función hasta que la promesa se resuelve o se rechaza; no bloquea todo JavaScript.

```
const response = await fetch(
  'https://randomuser.me/api/?results=50'
);
if (!response.ok) throw new Error('Error HTTP');
const data = await response.json();
setPersonas(data.results);
```

Qué hace cada paso

Expresión	Qué obtiene o hace
`await fetch(url)`	Espera una respuesta HTTP. El resultado es un objeto Response, todavía no la lista.
`response.ok`	Indica si el estado HTTP está entre 200 y 299. Un 404 o 500 no hace que fetch rechace automáticamente.
`await response.json()`	Lee el cuerpo e interpreta su JSON, produciendo un valor de JavaScript. También puede fallar.
`data.results`	En esta API, es el arreglo de objetos de usuarios. El nombre results pertenece a esa respuesta; no es universal.
`setPersonas(...)`	Guarda la lista y solicita un render para mostrarla.

Separar datos, carga y error

```
const [personas, setPersonas] = useState([]);
const [isLoading, setIsLoading] = useState(true);
const [error, setError] = useState('');
```

personas empieza como arreglo vacío para poder recorrerlo con `map`. **isLoading** indica si hay una carga en curso. **error** guarda el mensaje para mostrar cuando falla. Los tres representan cosas distintas.

En `try` se ejecuta la consulta; `catch` trata el fallo; `finally` termina la carga tanto si hubo éxito como si hubo error. Si solo se pone `setIsLoading(false)` al final del try, una falla anterior salta esa línea.

Conexión con el parcial. personas contiene un arreglo de objetos, no el JSON completo ni un texto. isLoading solo controla qué mostrar mientras se consulta; no almacena los usuarios ni corrige dependencias.

Para ampliar: Uso de fetch · Response.json · Funciones async

6. Ejemplo completo para relacionar todo

Carga al montar, gestión de errores y renderizado condicional

Este componente puede importarse y mostrarse desde App.jsx. Usa la misma API del parcial. El código agrega control de errores y evita guardar resultados de un efecto que ya se limpió.

```
import { useEffect, useState } from 'react';

export default function Usuarios() {
  const [personas, setPersonas] = useState([]);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState('');

  useEffect(() => {
    let activo = true;
    async function cargar() {
      setIsLoading(true);
      setError('');
      try {
        const response = await fetch(
          'https://randomuser.me/api/?results=50'
        );
        if (!response.ok) throw new Error('Error HTTP');
        const data = await response.json();
        if (activo) setPersonas(data.results);
      } catch (e) {
        if (activo) setError('No se pudo cargar la lista.');
      } finally {
        if (activo) setIsLoading(false);
      }
    }
    cargar();
    return () => { activo = false; };
  }, []);

  if (isLoading) return <p>Cargando...</p>;
  if (error) return <p role="alert">{error}</p>;
  if (personas.length === 0) return <p>No hay usuarios.</p>;

  return (
    <ul>
      {personas.map((p) => (
        <li key={p.login.uuid}>
          {p.name.first} {p.name.last}
        </li>
      ))}
    </ul>
  );
}
```

Cómo leerlo sin memorizarlo

Primero declara la memoria del componente. Después, el efecto inicia la consulta. Cuando cambia el estado, React renderiza otra vez y elige entre carga, error, lista vacía o resultados. `map` transforma cada usuario en un elemento de lista y `key` aporta una identidad estable para reconciliar esos elementos.

La variable activo pertenece a esa ejecución del efecto y su limpieza la cambia a false: así se ignoran resultados tardíos. Esto no cancela la petición de red. La función del efecto no es async; define y llama a cargar, porque debe poder devolver una función de limpieza, no una promesa.

Para ampliar: Efectos y consultas · Fetch